

MyBatis-Plus 深度开发规范手册

夏壹 夏壹分享 2026年4月3日 07:01 北京

01

核心调用链要求

强制：系统数据流转必须遵循 `Controller -> Service -> Mapper` 的单向调用链路，禁止跨层调用、逆向调用。

依赖管理要求

强制：严禁 Service 层出现循环依赖场景。
例如 A Service 调用 B Service，同时 B Service 又调用 A Service 的情况绝对不允许出现。

02

实体类（Entity）与数据库建模规范

公共字段维护要求

强制：禁止手动维护 `update_time`、`create_time`、`deleted`、`version` 等公共审

计字段，必须通过 MyBatis-Plus 自动填充机制实现。

敏感信息处理规范

规范：手机号、身份证号、密码等敏感数据，在 Entity 或 VO 层需通过 `@TableField(select = false)` 屏蔽默认查询，或结合自定义 Jackson 序列化器实现数据脱敏，避免敏感信息泄露。

03

事务管理规约（规避 10% 隐形 Bug）

事务边界要求

强制：事务必须声明在 Service 层公共方法上，禁止在 Controller 层或 Mapper 层声明事务。

事务传播特性要求

规范：多事务嵌套场景需明确指定传播特性（如 `REQUIRED`、`REQUIRES_NEW`），避免因传播特性滥用导致事务失效或数据不一致。

代码编写“六不准”（硬性军规）

不准跨层操作

不准跳过 Service 层，直接在 Controller 层调用 Mapper 层方法。

不准硬编码 SQL 片段

不准在代码中硬编码表名、字段名等 SQL 片段，必须通过 Entity 字段或枚举类统一管理。

不准忽略空指针

不准在 Lambda 条件查询时忽略入参 null 判断，必须使用 `.condition()` 方法做条件过滤。

不准禁用分页插件

不准关闭 MyBatis-Plus 分页插件，所有列表查询必须走分页逻辑，避免全表查询撑爆内存。

不准手动处理逻辑删除

不准手动拼接 `deleted = 0` 条件，必须依赖 MyBatis-Plus 逻辑删除插件自动处理。

不准忽略乐观锁

高并发更新场景不准忽略乐观锁（`version` 字段），避免数据覆盖（Lost Update）问题。

05

SQL 与性能优化规范（解决 5% 慢查询 Bug）

索引设计要求

规范：查询条件中高频使用的字段（如 `userId`、`orderNo`）必须建立索引，联合索引需遵循“最左匹配原则”。

大字段查询要求

规范：包含 `TEXT`、`BLOB` 等大字段的表，查询时需通过 `@TableField(select = false)` 屏蔽大字段，仅在需要时手动指定查询。

批量操作要求

规范：批量插入/更新数据时，必须使用 MyBatis-Plus 批量操作 API
(如 `saveBatch`)，单次批量操作数量控制在 500 条以内，避免数据库连接超时。

06

审计与扩展功能规范

自动填充实现规范

以下是公共字段自动填充的标准实现代码（含详细注解）：

```

package com.yourcompany.project.common.handler;

import com.baomidou.mybatisplus.core.handlers.MetaObjectHandler;
import lombok.extern.slf4j.Slf4j;
import org.apache.ibatis.reflection.MetaObject;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;

/**
 * 自定义元对象字段填充控制器
 * 实现公共字段（createTime/updateTime/deleted/version）的自动填充
 * 避免手动设置导致的字段遗漏、值不一致问题
 */
@Slf4j // 日志注解，方便排查填充逻辑问题
@Component // 注入Spring容器，让MyBatis-Plus识别到该处理器
public class MyMetaObjectHandler implements MetaObjectHandler {

    /**
     * 插入操作时的字段填充逻辑
     * @param metaObject 元对象（封装了实体类的属性和值）
     */
    @Override
    public void insertFill(MetaObject metaObject) {
        log.info("开始执行插入操作的字段自动填充....");
        // 填充创建时间（严格填充模式，确保字段未手动赋值时才填充）
        this.strictInsertFill(metaObject, "createTime", LocalDateTime.class, LocalDateTime.now());
        // 填充更新时间（新增时与创建时间保持一致）
        this.strictInsertFill(metaObject, "updateTime", LocalDateTime.class, LocalDateTime.now());
        // 填充逻辑删除标记（0=未删除，符合MyBatis-Plus逻辑删除默认约定）
        this.strictInsertFill(metaObject, "deleted", Integer.class, 0);
        // 填充乐观锁版本号（初始值为1）
        this.strictInsertFill(metaObject, "version", Integer.class, 1);
    }

    /**
     * 更新操作时的字段填充逻辑
     * @param metaObject 元对象
     */
    @Override
    public void updateFill(MetaObject metaObject) {
        log.info("开始执行更新操作的字段自动填充....");
    }

```

```
// 填充更新时间（每次更新都刷新为当前时间）
    this.strictUpdateFill(metaObject, "updateTime", LocalDateTime.class, LocalDateTime.now());
}
}
```

基础实体类封装规范

封装所有实体类通用字段，减少重复代码，统一规范（含详细注解）：



```

package com.yourcompany.project.common.entity;

import com.baomidou.mybatisplus.annotation.*;
import com.fasterxml.jackson.databind.annotation.JsonSerialize;
import com.fasterxml.jackson.databind.ser.std.ToStringSerializer;
import lombok.Data;
import java.time.LocalDateTime;

/**
 * 基础实体类
 * 所有业务实体类都需继承此类，统一管理公共字段
 */
@Data // Lombok注解，自动生成get/set/toString等方法
public abstract class BaseEntity {

    /**
     * 主键ID（雪花算法生成）
     * 使用ToStringSerializer序列化，解决前端Long类型精度丢失问题
     */
    @JsonSerialize(using = ToStringSerializer.class)
    @TableId(type = IdType.ASSIGN_ID) // 指定主键生成策略为雪花算法
    private Long id;

    /**
     * 创建时间（由MyMetaObjectHandler自动填充）
     */
    @TableField(fill = FieldFill.INSERT) // 仅插入时填充
    private LocalDateTime createTime;

    /**
     * 更新时间（由MyMetaObjectHandler自动填充）
     */
    @TableField(fill = FieldFill.INSERT_UPDATE) // 插入和更新时填充
    private LocalDateTime updateTime;

    /**
     * 乐观锁版本号（用于解决高并发数据覆盖问题）
     */
    @Version // MyBatis-Plus乐观锁注解
    @TableField(fill = FieldFill.INSERT) // 仅插入时填充初始值
    private Integer version;

```



```
/**
 * 逻辑删除标记（0=未删除，1=已删除）
 */
@TableLogic // MyBatis-Plus逻辑删除注解
@TableField(fill = FieldFill.INSERT) // 仅插入时填充初始值
private Integer deleted;
}
```

07

投产前 Check List（拦截最后 5% 的 Bug）

- **空指针检查**： `LambdaQuery` 构建查询条件前，是否对入参做了 `null` 判断？（或使用 `.condition()` 方法做条件过滤）；
- **分页溢出检查**：是否配置 `PaginationInnerInterceptor` 分页插件，且设置最大单页限制（建议 500 条）？；
- **字段对应检查**：Entity 中的字段在数据库中是否都存在？（不存在的字段必须添加 `@TableField(exist = false)` 注解）；
- **日志检查**：生产环境配置文件（ `application-prod.yml` ）是否关闭 SQL 日志打印？（防止日志文件过大撑爆磁盘）；
- **缓存同步检查**：若使用 Redis 缓存，执行 `updateById` 等更新操作后，是否同步删除对应缓存？（避免缓存脏数据）；

全量配置规范实现

MyBatis-Plus 核心插件配置（含详细注解）

集成分页、乐观锁、防全表操作插件，保障大数据量下的系统安全与性能：



```

package com.yourcompany.project.common.config;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.BlockAttackInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.OptimisticLockerInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * MyBatis-Plus 核心配置类
 * 集成分页、乐观锁、防全表操作等核心插件，统一配置管理
 */
@Configuration // 标记为Spring配置类
@MapperScan("com.yourcompany.project.mapper") // 扫描Mapper接口所在包，替代xml配置
public class MybatisPlusConfig {

    /**
     * 构建MyBatis-Plus插件链
     * @return MybatisPlusInterceptor 插件拦截器
     */
    @Bean // 将插件链注入Spring容器
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();

        // 1. 分页插件配置（适配MySQL数据库）
        PaginationInnerInterceptor paginationInterceptor = new PaginationInnerInterceptor();
        // 设置单页最大查询条数，防止一次性查询过多数据撑爆JVM内存
        paginationInterceptor.setMaxLimit(500L);
        interceptor.addInnerInterceptor(paginationInterceptor);

        // 2. 乐观锁插件配置（解决高并发数据覆盖问题）
        interceptor.addInnerInterceptor(new OptimisticLockerInnerInterceptor());

        // 3. 防全表操作插件（禁止无条件的全表删除/更新，防止误操作）
        interceptor.addInnerInterceptor(new BlockAttackInnerInterceptor());

        return interceptor;
    }
}

```

配置类核心作用说明

- **PaginationInnerInterceptor**：限制单页最大查询条数，避免大数据量查询导致 JVM 内存溢出；
- **OptimisticLockerInnerInterceptor**：实现乐观锁机制，解决高并发场景下“先读后写”导致的数据覆盖问题；
- **BlockAttackInnerInterceptor**：拦截无条件的全表删除/更新操作，防止因代码 Bug 或运维误操作导致的数据全量丢失；
- 基础实体类中 **@JsonSerialize** 注解：解决雪花算法生成的 Long 类型 ID 在前端展示时的精度丢失问题；
- 自动填充处理器：保证所有数据的审计字段（createTime/updateTime 等）完整且一致，实现全链路数据追踪。

规范价值说明

制定并落地这些规范，不是为了限制开发效率，而是为了从根源上规避生产环境的高频问题，让开发者不用在凌晨 3 点爬起来处理线上事故，保障系统的稳定性和可维护性。

推荐文章：

1. [CICD+Docker+Dockerfile三大系列37篇系统讲解](#)
2. [《剑指Offer》刷题笔记66篇](#)
3. [RabbitMQ+MySQL30篇两大系列讲解](#)